



**ITM SKILLS
UNIVERSITY**

FINAL PROJECT REPORT

Bachelor of Technology in Computer Science and Engineering

Class BTech CSE 25-29

PROJECT TITLE

Student Attendance Management System

Submitted By	Sanjana Bashya
---------------------	----------------

Roll No:	150096725113
-----------------	--------------

Cohort:	Larry Page
----------------	------------

Subject	Java Programming
----------------	------------------

1. Project overview

This project is a console application written in Java that keeps track of student attendance. Instead of noting attendance in a register, the user runs the program, registers the students, and enters how many classes each of them attended. The program works out the attendance percentage and shows whether the student meets the required limit.

Apart from recording attendance, the program can search for a student by roll number and print a report of all registered students. While building it I used the basic Java topics we studied: arrays, loops, methods, the Scanner class, operators, if-else statements, switch-case and searching.

2. Problem statement

An educational institute needs a simple way to record attendance and calculate each student's percentage. The system should store the attendance records, calculate the percentage, point out students who are below the required percentage, print an attendance report, and let the user look up one student at a time.

The required attendance used in this project is 75%.

3. Objectives

- Store student details in arrays.
- Record the total classes held and the classes attended for every student.
- Calculate the attendance percentage.
- Check whether a student is eligible under the 75% rule.
- Generate an attendance report for all students.
- Search for a student using the roll number.
- Practise the basic Java concepts by using them in a working program.
- Keep the program easy to use with a simple menu.

4. Scope of the project

The project covers basic attendance management for a class or a small group. It can hold up to 50 students. For each student it can register them, save their attendance, calculate the percentage, check eligibility, search their record and show a complete report.

It is meant to show how the fundamental Java concepts fit together, so it is not a full attendance system. The data is kept in arrays in memory, which means it is lost when the program is closed. There is no database, no login and no graphical interface.

5. System modules

The program is divided into six modules. Each one is handled by a separate method.

5.1 Student registration

This module adds a new student. The user enters a roll number and a name, and both are stored in the arrays. If the roll number is already registered, the program refuses it, so two students cannot share a roll number.

5.2 Attendance entry

Here the user enters the roll number of a registered student, then the total number of classes and the number of classes attended. The program checks that the attended classes are not more than the total classes before saving anything.

5.3 Percentage calculation

The percentage is calculated with this formula:

$$\text{Attendance Percentage} = (\text{Classes Attended} / \text{Total Classes}) \times 100$$

For example, if a student attended 32 out of 40 classes, the attendance percentage is $(32 / 40) \times 100 = 80\%$.

5.4 Eligibility check

The program compares the percentage with the required 75%.

Condition	Result
Attendance Percentage $\geq$ 75%	Eligible
Attendance Percentage $<$ 75%	Not Eligible

5.5 Attendance report

This module prints one row for every registered student with the roll number, name, attendance percentage and eligibility status.

5.6 Student search

The user types a roll number and the program looks for it in the roll number array with a for loop. If it is found, the student's details are displayed. If not, the program prints "Student not found."

6. Data structures used

All student data is stored in arrays of size 50. The arrays run in parallel, which means that position *i* in every array belongs to the same student. For example, `names[2]`, `rolls[2]`, `totalClasses[2]` and `attendedClasses[2]` are all about the third registered student.

Array	Declaration	Stores
<code>names</code>	<code>static String[] names = new String[50];</code>	Student names
<code>rolls</code>	<code>static int[] rolls = new int[50];</code>	Roll numbers
<code>totalClasses</code>	<code>static int[] totalClasses = new int[50];</code>	Total classes held
<code>attendedClasses</code>	<code>static int[] attendedClasses = new int[50];</code>	Classes attended

A separate variable, `count`, keeps the number of students registered so far. It also tells the program where the next student should be stored.

7. Java concepts used

Concept	How it is used in this project
Arrays	Four arrays store the names, roll numbers, total classes and attended classes of up to 50 students.
Scanner	<code>static Scanner sc = new Scanner(System.in);</code> reads the menu choice, roll numbers, names and class counts typed by the user.
Methods	<code>registerStudent()</code> , <code>enterAttendance()</code> , <code>getPercentage()</code> , <code>search()</code> , <code>showReport()</code> and <code>main()</code> split the program into small parts. Two helper methods, <code>readInt()</code> and <code>findIndex()</code> , avoid repeating code.
Loops	A for loop searches for a roll number and prints the report. A do-while loop keeps showing the menu until the user chooses Exit.
If-else statements	Used to check whether a student exists, to validate the attendance, to decide eligibility, and to handle the case of no registered students.
Switch-case	Handles the five menu options and an invalid choice.
Operators	Arithmetic (<code>/</code> and <code>*</code>) for the percentage, and comparison operators (<code>==</code> , <code>!=</code> , <code>></code> , <code>>=</code> , <code><</code>) for the checks.

8. Algorithm

1. Start the program.
2. Display the main menu.
3. Ask the user to select an option.

4. If the user selects registration, take the roll number and name and store them.
5. If the user selects attendance entry, search for the student using the roll number.
6. Take the total classes and the classes attended.
7. Calculate the attendance percentage.
8. Compare the percentage with 75%.
9. Display whether the student is Eligible or Not Eligible.
10. If the user asks for the report, display the details of all registered students.
11. If the user asks to search, find the student by roll number and display the record.
12. Repeat from step 2 until the user selects Exit.
13. End the program.

9. Output screenshots

The screenshots below show the program running. The inputs used for each screen are written above the picture. Figures 3 to 8 are taken in the same run, after registering the three students in Figure 2.

```
===== STUDENT ATTENDANCE MANAGEMENT =====
1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit
Enter your choice: 1
Enter Roll Number: 101
Enter Student Name: Apoorva
Student registered successfully!

===== STUDENT ATTENDANCE MANAGEMENT =====
1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit
Enter your choice: 1
Enter Roll Number: 102
Enter Student Name: David
Student registered successfully!

===== STUDENT ATTENDANCE MANAGEMENT =====
1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit
Enter your choice: 1
Enter Roll Number: 103
Enter Student Name: Harini
Student registered successfully!
```

===== STUDENT ATTENDANCE MANAGEMENT =====

1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit

Enter your choice: 1
Enter Roll Number: 104
Enter Student Name: Isha
Student registered successfully!

===== STUDENT ATTENDANCE MANAGEMENT =====

1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit

Enter your choice: 1
Enter Roll Number: 105
Enter Student Name: Lily
Student registered successfully!

===== STUDENT ATTENDANCE MANAGEMENT =====

1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit

Enter your choice: 2
Enter Roll Number: 101
Enter Total Classes: 45
Enter Classes Attended: 39
Attendance saved successfully.

===== STUDENT ATTENDANCE MANAGEMENT =====

1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit

Enter your choice: 3

Enter Roll Number: 101

Attendance Percentage: 86.67%

===== STUDENT ATTENDANCE MANAGEMENT =====

1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit

Enter your choice: 4

Enter Roll Number: 101

Student is Eligible.

===== STUDENT ATTENDANCE MANAGEMENT =====

1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit

Enter your choice: 5

----- ATTENDANCE REPORT -----

Roll Number: 101

Name: Apoorva

Attendance: 86.67%

Status: Eligible

Roll Number: 102

Roll Number: 102
Name: David
Attendance: 0.00%
Status: Not Eligible

Roll Number: 103
Name: Harini
Attendance: 0.00%
Status: Not Eligible

Roll Number: 104
Name: Isha
Attendance: 0.00%
Status: Not Eligible

Roll Number: 105
Name: Lily
Attendance: 0.00%
Status: Not Eligible

===== STUDENT ATTENDANCE MANAGEMENT =====

1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit

Enter your choice: 6
Enter Roll Number: 101
Student Name: Apoorva
Attendance: 86.66666666666667%

===== STUDENT ATTENDANCE MANAGEMENT =====

1. Register Student
2. Add Attendance
3. Check Attendance Percentage
4. Check Eligibility
5. Attendance Report
6. Search Student
7. Exit

Enter your choice: 7
Thank you! Program ended.

10. Validation and error handling

The program checks the input so that wrong values do not end up in the records. If the user enters 40 total classes and 45 attended classes, it shows:

Error: Attended classes cannot be more than total classes.

The other checks are listed below.

Situation	What the program does
A roll number that is not registered is searched	Displays "Student not found."
The report is requested with no students	Displays "No students registered yet."
A roll number is registered twice	Displays "Error: This roll number is already registered."
Total classes is zero or a number is negative	Displays an error and does not save the values
A letter is typed where a number is needed	Displays "Please enter a valid number." and asks again
A menu option other than 1 to 5	Displays "Invalid choice. Please select 1 to 5."
A student has no attendance entered yet	Shows "Not entered" in the report instead of a percentage

11. Testing

I tested the program with different inputs and compared what it displayed with what I expected. All the test cases passed.

No	Test case	Input	Expected result	Result
1	Student registration	Roll 101, Sanjana	Student registered	Pass
2	Attendance entry	40 total, 32 attended	Attendance saved	Pass
3	Percentage	32 / 40	80.00%	Pass
4	Eligibility	80%	Eligible	Pass
5	Low attendance	20 / 40	Not Eligible	Pass
6	Invalid attendance	45 attended, 40 total	Error message	Pass
7	Search existing student	Roll 101	Student details displayed	Pass
8	Search unknown student	Roll 999	Student not found	Pass
9	Empty report	No students	No students registered	Pass
10	Duplicate roll number	Roll 101 twice	Error message	Pass

No	Test case	Input	Expected result	Result
11	Wrong menu input	9, then abc	Error message, menu shown again	Pass

12. Future enhancements

The project can be improved in several ways:

- Connect it to a database such as MySQL or PostgreSQL so the records are not lost when the program closes.
- Build a graphical user interface.
- Add a login for teachers and administrators.
- Record attendance date by date instead of only the totals.
- Export the report to PDF or Excel.
- Keep more details in each student profile.
- Send a notification to students who fall below the required percentage.
- Store the records in the cloud so they can be opened from different devices.

13. Conclusion

The Student Attendance Management System shows how the Java basics can be used to solve a small real problem. Arrays hold the student records, Scanner takes the input, loops handle the searching and the menu, methods keep the code organised, operators do the percentage calculation, and if-else statements decide eligibility.

One thing I paid attention to was the Scanner. After reading a number with `nextInt()`, the newline stays in the input, which can cause the next name to be skipped. The `readInt()` method clears it, and it also asks again when the user types something that is not a number.

The program does everything the problem statement asks for: it stores attendance, calculates percentages, identifies students below 75%, prints a report and finds a student by roll number. It is a simple console program, but it gave me good practice in putting the different Java concepts together.

14. Source Code

Github Link- <https://github.com/sanjana123642/Student-Attendance-Management-System.git>